MarketSage AI

Master Engineering Blueprint — **Part V**

Backtesting, Evaluation and MLOps

Three levels of simulation, the event-driven simulator, the full cost model, the metric hierarchy, attribution, environments, operating rhythm and observability.

This is an extract of the MarketSage AI master plan, published separately for convenience. Chapter, section and equation numbering matches the full document, and cross-references point back into it. Read Chapter 3 (Reality Calibration) first — every design decision in this part follows from it.

CHAPTER 13

Backtesting and Simulation

13.1 Three levels of simulation

Level	What it is	What it can and cannot tell you
L1 Vectorised	Fast pandas/DuckDB evaluation over historical features and labels	Fast model iteration and feature screening. **Cannot** model execution, latency or queue position. Never trust a P&L number from here.
L2 Event-driven	A simulator that replays ticks, news and chain snapshots in timestamp order through the actual production code path	The real answer. Models latency, partial fills, spread, stale quotes, exits. Slow but honest.
L3 Live paper	The production system running on live data with simulated fills	Catches everything L2 misses: real feed hiccups, real API errors, real timing. **The only evidence that counts for the Phase 7 gate.**

Rule

Model research happens at L1. *Every claim about profitability* must come from L2, and *every go-live decision* must come from L3. A number produced at L1 may never appear in a slide about returns.

13.2 The event-driven simulator

Listing 13.1: Simulator core loop (conceptual)

```
# Single merged, timestamp-ordered stream. This ordering IS the correctness
# guarantee: nothing can be consumed before its t_available.
for msg in merged_stream(ticks, chain_snaps, news, calendar, start, end):
    clock.set(msg.ts)
    bus.publish(msg) # same bus code as production
    for out in engine.drain(): # same engine code as production
        if out.kind == "reco":
            broker_sim.submit(out, latency=sample_latency(),
                              slippage=slippage_model(out, book_state))
    broker_sim.mark_to_market(clock.now())
    risk.check(clock.now())
```

Realism requirement	Implementation
Decision latency	Inject a sampled delay between event timestamp and the moment the engine is allowed to act. Draw from the measured live latency distribution, not a constant.
Fill model	Marketable orders fill at the far touch plus a slippage term that grows with size relative to displayed depth and with spread width. Limit orders fill only if the market trades through them, with a queue-position discount.
Partial fills	Multi-lot orders may partially fill; the engine must handle it.

Stale quotes	If the last chain snapshot is older than x seconds, the simulator refuses the trade, exactly as production must.
Costs	Full stack: brokerage, exchange transaction charge, STT/CTT, SEBI fee, GST, stamp duty, plus modelled slippage. Rates change — keep them in a dated configuration table, applied point-in-time.
Impact	For retail size in index options, market impact is small but not zero; model it as a function of order size relative to trailing volume.
Exits	Simulated with intrabar path assumptions that are *conservative*: if both target and stop are touched within the same minute, assume the stop filled first.

The conservative-tie rule

The stop-first assumption on ambiguous bars typically reduces backtest returns by a large margin. Keep it. Every optimistic assumption you allow is a loan you repay with real money later.

13.3 Cost model

Component	Applies to	Modelling note
Brokerage	Per executed order	Flat per order for most discount brokers; a two-leg spread entered and exited is four orders.
Exchange transaction charge	Premium turnover	Percentage of premium.
Securities transaction tax	Sell side (and exercised options)	Materially different on expiry if allowed to expire in the money — **always square off, never let ITM options expire.**
Regulator fee, stamp duty	Turnover	Small but included.
GST	On brokerage plus charges	Included.
Slippage	Every fill	The dominant real-world cost. Estimate from your own live paper fills, not from theory.

The breakeven reality check

Compute your all-in round-trip cost as a percentage of the option premium at entry. For small premiums and multi-leg structures this can be several percent. The engine must require $\mathbb{E}[\text{P\&L}]$ to exceed this by a comfortable margin — otherwise the strategy is a machine for converting edge into fees.

CHAPTER 14

Evaluation Framework

14.1 Metric hierarchy

Metrics are organised by what decision they inform. Do not mix layers.

Layer	Metrics	Decision it drives
Detection	Precision, recall, macro-F1 per event class; first-mention latency; duplicate rate; false-positive rate on the noise class	Should the classifier ship?
Impact	Coverage of predicted intervals; correlation of predicted and realised abnormal return; sample sizes	Is the impact library trustworthy?
Forecast	Brier score and decomposition; reliability slope and intercept; log loss; AUC; pinball loss; CRPS; PIT uniformity	Is the model calibrated?
Decision	Hit rate, payoff ratio, expectancy per trade, edge realisation (realised versus predicted edge)	Is the decision layer converting forecasts into good trades?
Portfolio	Net P&L after costs, profit factor, Sharpe, Sortino, max drawdown, Calmar, exposure time, turnover, cost drag	Should we scale capital?
Operational	p50/p99 latency per stage, uptime during market hours, data completeness, error budget	Is the system healthy?

14.2 The attribution table — the most useful report you will build

Every trade is tagged with its event class, regime, time bucket, structure type, horizon and model version. The weekly report then answers questions such as:

Event class	n	Hit %	Avg R	Net	Verdict
monetary.rate_decision	12	66.7	+0.81	positive	Keep, size up
macro.inflation_release	19	57.9	+0.34	positive	Keep
geopolitical.rhetoric	31	45.2	−0.22	negative	**Disable this class**

energy.supply_disruption	7	57.1	+0.11	flat	Insufficient sample
flows.expiry_pinning	14	71.4	+0.62	positive	Keep, investigate scaling

Class-level kill policy

Any event class with at least 25 trades and negative expectancy over a rolling quarter is **automatically disabled** by the system, not by a human debate. Re-enabling requires a documented model change and a fresh out-of-sample period. This single policy does more for long-run profitability than any model improvement.

14.3 The public accuracy dashboard, done properly

The original dashboard shows "32/47 correct (68%)". Replace with:

Panel	Content
Reliability curve	Predicted probability bucket versus realised frequency, with confidence bands and the count in each bucket.
Rolling Brier skill score	Versus a climatological baseline; the honest measure of whether the model adds information.
Interval coverage	Realised coverage of the 50% and 90% bands over the trailing 20 sessions.
Edge realisation	Scatter of predicted edge versus realised edge per trade; the regression slope should be near 1. A slope near 0 means the model has no usable edge regardless of hit rate.
Cost drag	Gross versus net P&L, so cost erosion is always visible.
Drawdown curve	Underwater plot with the current position on the risk ladder.
Latency histogram	Per stage, p50/p95/p99, refreshed live.
Per-class attribution	The table above, always visible.

CHAPTER 15

MLOps, Infrastructure and Operations

15.1 Environments

Env	Data	Purpose
`research`	Historical Parquet	Notebooks, feature exploration, model training. No connection to any broker credential, ever.
`replay`	Recorded live days	Deterministic regression testing of the full pipeline. CI runs a fixed golden day on every commit.
`paper`	Live feeds	Production code, simulated fills. Runs every session from Phase 5 onward.
`live`	Live feeds	Real orders. Separate credentials, separate machine, separate alerting, stricter limits.

15.2 The daily operating rhythm

Time (IST)	Job	Detail
06:30	Overnight digest	Collect global overnight events, compute the gap forecast, publish the pre-market briefing.
08:45	Preflight	Feed connectivity, clock sync, model version check, margin check, calendar check (holiday, expiry, special session). Fails loudly.
09:00–09:15	Pre-open watch	Observe only; no signals.
09:15–15:30	Live session	Hot and cold lanes active; risk gate armed.
15:20	Forced flatten	All intraday positions squared off; no exceptions.
15:45	Post-session	Compute outcomes for every prediction, update the ledger, run attribution, push metrics.
18:00	Nightly	Feature materialisation, data-quality gates, archive to Parquet, backup verification.
Weekend	Retrain	Incremental retrain, walk-forward, champion/challenger evaluation, calibration refresh, weekly report generation.

15.3 Observability

- **Trace ID propagation.** One identifier flows from the raw news item through classification, features, prediction, recommendation and fill. Any recommendation can be reconstructed end to end in one query. This is non-negotiable for debugging a system with this many stages.
- **Golden signals per plane:** latency, throughput, error rate, saturation.
- **Business alarms:** no signal emitted for two consecutive sessions (system silently broken), signal rate three times the normal rate (classifier gone haywire), calibration slope outside $[0.8, 1.2]$, feature null rate spike.

- **Every prediction is logged with its feature vector hash** so that post-hoc analysis can recompute exactly what the model saw.

15.4 Reproducibility

Artifact	Discipline
Code	Git; every model artifact records the commit hash.
Data	Immutable, dated Parquet partitions; a dataset version is a manifest of file hashes.
Features	Versioned definitions; changing a feature's logic requires a version bump, never an in-place edit.
Models	MLflow registry with stage transitions (staging, shadow, production, archived). Production reads only from the registry.
Config	All thresholds in versioned YAML, loaded at startup, hash logged. No magic numbers in code.
Randomness	Seeded everywhere; the simulator is deterministic given a seed.

15.5 Security

- Broker credentials and API keys in a secrets manager, never in the repository, never in a notebook, never in an environment file committed by accident. Add a pre-commit secret scanner on day one.
- The live trading process runs on a separate host with a minimal attack surface and no inbound ports except an authenticated control channel.
- Two-person rule (or a written checklist plus a delay) for changing risk limits.
- All order-placing code paths require an explicit `live=True` argument that defaults to `False` in every environment.
- Immutable audit log of every order intent, shipped off-host.